

Pandas — LEVEL 7: Data Types

Line-by-line code + output + examples + easy interview explanations

1. What is dtype?

Code

```
import pandas as pd

df = pd.DataFrame({
    "Age": [22, 24, 23],
    "Salary": [30000.5, 40000.0, 35000.5],
    "Name": ["Amit", "Riya", "John"]
})

print(df.dtypes)
```

Output

```
Age          int64
Salary       float64
Name         object
dtype: object
```

Explanation: dtype means data type. It tells pandas what type of data is stored in a column.

Easy interview explanation: dtype tells us the type of data stored in a pandas Series or DataFrame column.

2. Integer

Code

```
df = pd.DataFrame({"Age": [22, 24, 23]})

print(df["Age"])
print(df["Age"].dtype)
```

Output

```
0    22
1    24
2    23
Name: Age, dtype: int64
int64
```

Explanation: Integer contains whole numbers without decimal values, such as 10, 25, and 100.

Easy interview explanation: Integer is commonly used for age, quantity, and counts.

3. Float

Code

```
df = pd.DataFrame({"Salary": [30000.5, 40000.75, 35000.25]})

print(df["Salary"])
print(df["Salary"].dtype)
```

Output

```
0    30000.50
1    40000.75
2    35000.25
Name: Salary, dtype: float64
```

```
float64
```

Explanation: Float contains numbers with decimal values, such as 10.5 and 99.99.

Easy interview explanation: Float is commonly used for salary, price, percentage, and measurements.

4. String

Code

```
df = pd.DataFrame({"Name": ["Amit", "Riya", "John"]})

print(df["Name"])
print(df["Name"].dtype)
```

Output

```
0    Amit
1    Riya
2    John
Name: Name, dtype: object
object
```

Explanation: Text data is commonly stored as object in pandas DataFrames. Pandas also has a dedicated string dtype.

Easy interview explanation: String data contains text such as names, cities, and departments.

5. Boolean

Code

```
df = pd.DataFrame({"Passed": [True, False, True]})

print(df["Passed"])
print(df["Passed"].dtype)
```

Output

```
0     True
1    False
2     True
Name: Passed, dtype: bool
bool
```

Explanation: Boolean has two logical values: True and False.

Easy interview explanation: Boolean is useful for flags such as Passed, Active, and Eligible.

6. Object

Code

```
df = pd.DataFrame({"Data": ["Amit", 25, "Pune"]})

print(df["Data"])
print(df["Data"].dtype)
```

Output

```
0    Amit
1     25
2    Pune
Name: Data, dtype: object
object
```

Explanation: object is a general-purpose pandas dtype. It is commonly seen with text and can also hold mixed Python objects.

Easy interview explanation: Object is a general dtype; use a more specific dtype when appropriate.

7. Category

Code

```
df = pd.DataFrame({
    "Department": ["IT", "HR", "IT", "Finance"]
})

df["Department"] = df["Department"].astype("category")

print(df["Department"])
print(df["Department"].dtype)
```

Output

```
0      IT
1      HR
2      IT
3  Finance
Name: Department, dtype: category
Categories (3, object): ['Finance', 'HR', 'IT']
category
```

Explanation: Category is useful when a column contains a limited number of repeated values.

Easy interview explanation: Category can reduce memory usage and is useful for repeated categorical values.

8. Datetime

Code

```
df = pd.DataFrame({
    "Date": ["2026-01-10", "2026-02-15", "2026-03-20"]
})

df["Date"] = pd.to_datetime(df["Date"])

print(df["Date"])
print(df["Date"].dtype)
```

Output

```
0    2026-01-10
1    2026-02-15
2    2026-03-20
Name: Date, dtype: datetime64[ns]
datetime64[ns]
```

Explanation: Datetime stores dates and times in a format pandas can analyze.

Easy interview explanation: Use `pd.to_datetime()` when you need to work with dates and times.

9. dtypes

Code

```
df = pd.DataFrame({
    "Age": [22, 24, 23],
    "Salary": [30000.5, 40000.0, 35000.5],
    "Passed": [True, False, True],
})
```

```

    "Name": ["Amit", "Riya", "John"]
})

print(df.dtypes)

```

Output

```

Age          int64
Salary       float64
Passed        bool
Name          object
dtype: object

```

Explanation: dtypes shows the data type of every column in a DataFrame.

Easy interview explanation: Use df.dtypes to quickly inspect all column data types.

10. astype()

Code

```

df = pd.DataFrame({"Age": ["22", "24", "23"]})

print(df.dtypes)

df["Age"] = df["Age"].astype(int)

print(df.dtypes)
print(df)

```

Output

```

Age          object
dtype: object
Age          int64
dtype: object
   Age
0    22
1    24
2    23

```

Explanation: astype() converts a column to a specified dtype. Here, text numbers become integers.

Easy interview explanation: Use astype() when you know exactly which dtype you want.

11. to_numeric()

Code

```

df = pd.DataFrame({"Salary": ["30000", "40000", "35000"]})

df["Salary"] = pd.to_numeric(df["Salary"])

print(df)
print(df["Salary"].dtype)

```

Output

```

   Salary
0    30000
1    40000
2    35000
int64

```

Explanation: pd.to_numeric() converts values into numeric data.

Easy interview explanation: Use to_numeric() when numbers have been imported or stored as text.

12. to_numeric() with errors='coerce'

Code

```
df = pd.DataFrame({"Marks": ["90", "85", "ABC"]})

df["Marks"] = pd.to_numeric(df["Marks"], errors="coerce")

print(df)
```

Output

```
      Marks
0    90.0
1    85.0
2     NaN
```

Explanation: errors='coerce' converts invalid values to NaN instead of raising an error.

Easy interview explanation: This is useful for messy real-world numeric data.

13. convert_dtypes()

Code

```
df = pd.DataFrame({
    "Age": [22, 24, 23],
    "Name": ["Amit", "Riya", "John"],
    "Passed": [True, False, True]
})

df = df.convert_dtypes()

print(df.dtypes)
```

Output

```
Age          Int64
Name         string
Passed        boolean
dtype: object
```

Explanation: convert_dtypes() tries to convert columns to suitable pandas nullable dtypes.

Easy interview explanation: It is useful when you want better pandas dtypes with support for missing values.

14. astype() vs to_numeric() vs convert_dtypes()

Code

```
# Convert to a specific type
df["Age"] = df["Age"].astype(int)

# Convert values to numeric
df["Salary"] = pd.to_numeric(df["Salary"])

# Automatically choose suitable pandas dtypes
df = df.convert_dtypes()
```

Output

```
astype()      -> specified dtype
to_numeric()  -> numeric dtype
convert_dtypes() -> suitable pandas dtype
```

Explanation: These methods solve different conversion problems.

Easy interview explanation: Interviewers often ask the difference between these three methods.

15. Complete Data Types Example

Code

```
import pandas as pd

df = pd.DataFrame({
    "Age": [22, 24, 23],
    "Salary": [30000.5, 40000.0, 35000.5],
    "Name": ["Amit", "Riya", "John"],
    "Passed": [True, False, True],
    "Department": ["IT", "HR", "IT"],
    "JoiningDate": ["2026-01-10", "2026-02-15", "2026-03-20"]
})

df["Department"] = df["Department"].astype("category")
df["JoiningDate"] = pd.to_datetime(df["JoiningDate"])

print(df.dtypes)
```

Output

```
Age                int64
Salary             float64
Name               object
Passed             bool
Department         category
JoiningDate        datetime64[ns]
dtype: object
```

Explanation: This example puts the most common pandas data types together in one DataFrame.

Easy interview explanation: Correct data types make filtering, calculations, date operations, and cleaning more reliable.

LEVEL 7 QUICK REVISION

dtype	-> Data type
int	-> Whole numbers
float	-> Decimal numbers
string/object	-> Text
bool	-> True / False
category	-> Repeated categorical values
datetime64[ns]	-> Date and time
dtypes	-> Shows every column's dtype
astype()	-> Convert to a specified dtype
to_numeric()	-> Convert values to numeric
convert_dtypes()	-> Choose suitable pandas dtypes

Important interview point: Always check `df.dtypes` before calculations, filtering, date operations, or data cleaning.